

REPORT: ORCHESTRA-BACKEND REPOSITORY

Report Period: May 25 - June 15, 2026

EXECUTIVE SUMMARY

This report documents the individual contributions, engineering activities, and developmental progress within the orchestra-backend repository during its initial three-week development cycle. The project comprises a Python-based backend architecture designed for distributed task orchestration and workflow automation. Through an analysis of version control metadata, commit histories, and system artifacts, this document outlines the core responsibilities of the development team and evaluates the technical trajectory of the system.

1. INTRODUCTION

The orchestra-backend development under the GitHub organization **orchestra-A** serving as the central orchestration engine for the platform, the repository has undergone rapid iteration over a 20-day period. This report provides a formal assessment of individual engineering output, methodologies employed, and strategic recommendations for upcoming development phases. The scope of this analysis covers code architectures, pull request (PR) management, backend data migrations, and validation frameworks executed by the project's engineering team. The Team:

Name	Role	GitHub
Mitaali Singh	Lead · PM · AI Developer	@mitaalisingh
Naman Gupta	Knowledge Graph Engineer	@Naman-GG
Arnav Tripathi	Data Pipeline Engineer	@ArnavXT
Sarvagya Prakash	Infrastructure Engineer	@SarvagyaPrakash
Prince Negi	Interactive Canvas Specialist	@prince1111111111
Isha Mahadev	Interface Developer	@IshaMahadev

2. CONTEXT AND PROJECT OVERVIEW

2.1 Codebase Composition

The repository reflects a production-oriented Python ecosystem with the following baseline technical specifications:

- **Primary Language:** Python (98.1% of codebase)
- **Deployment Status:** Active continuous integration (CI) via Render cloud infrastructure

2.2 Structural Architecture

The backend implementation isolates core functional domains across a modular subsystem design:

1. **main.py**: Entry point hosting application initialization, RESTful API endpoints, and request routing middleware.
2. **models.py & models_sql.py**: Data model definitions managing the structural transition from transient memory states to relational database schemas.
3. **database.py**: Connection pooling and transactional initialization layer handling relational storage operations.
4. **state_machine.py**: Core workflow engine controlling task lifecycle transitions and automated validation states.
5. **scheduler.py**: Job queue scheduling, processing intervals, and asynchronous execution handlers.
6. **normalizer.py**: Sanitization and path standardization utilities feeding downstream graphing services.
7. **Test Suite Ecosystem**: Includes test_normalizer.py, test_scheduler.py, test_webhook.py, and test_history.py.

Service	URL
AI Server	https://orchestra-ai-36zm.onrender.com
Backend	https://orchestra-backend-2v5a.onrender.com
Docs	https://orchestra-ai-36zm.onrender.com/docs

3. OBJECTIVES

3.1 Primary Infrastructure Objectives

- Construct a resilient, state-driven backend architecture for secure task execution.
- Migrate system storage from mock in-memory constructs to a persistent **PostgreSQL** layer.
- Expose comprehensive **REST APIs** for predictable integration with frontend clients.
- Establish an immutable, auditable distributed task history tracking module.
- Secure cross-origin operations via configurable **Cross-Origin Resource Sharing (CORS)** middleware.

3.2 Near-Term Technical Targets

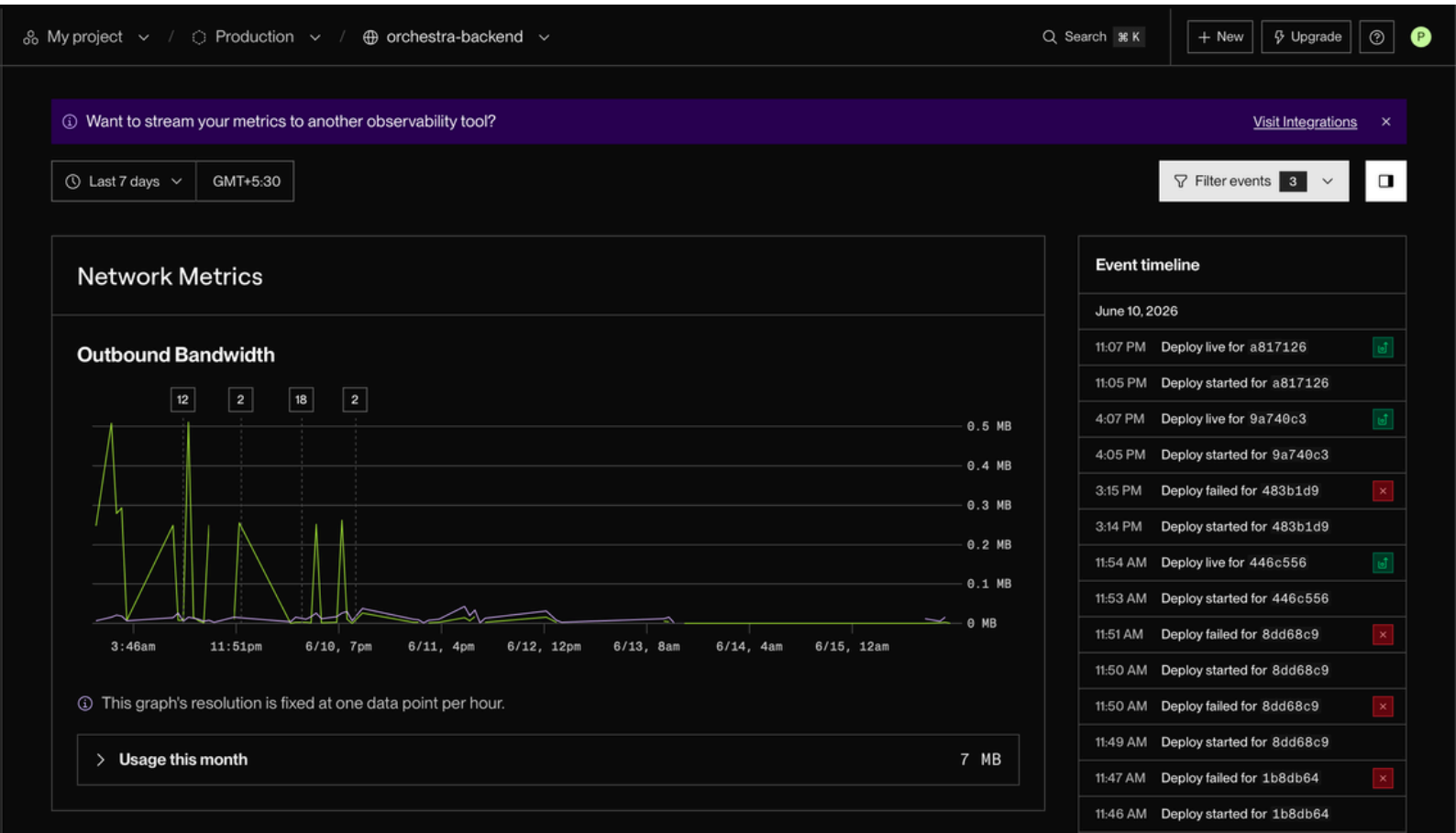
- Standardize API response structural contracts across all dynamic endpoints.
- Ensure structural compliance with the target cloud runtime engine.
- Incorporate reverse-proxy pipeline routing to forward analytical data smoothly to external AI graph layers.

4. METHODOLOGY

4.1 Engineering Workflow

The project utilizes an agile, trunk-based branching strategy prioritizing isolated feature development and mandatory peer reviews before production main integrations:

[Feature/Fix Branch] ---> [PR & Review] ---> [Merge to Main] ---> [Render Deployment]



Timeline Orchestra Backend

0.7.0 OAS 3.1

/openapi.json

Infrastructure layer for Timeline Orchestra

default			^
GET	/	Health Check	▼
POST	/webhook	Receive Webhook	▼
POST	/test/simulate-webhook	Simulate Webhook	▼
POST	/webhook/github	Receive Github	▼
POST	/webhook/discord	Receive Discord	▼
POST	/webhook/figma	Receive Figma	▼
GET	/tasks	Get Tasks	▼
POST	/tasks	Create New Task	▼
GET	/graph	Get Graph	▼
GET	/tasks/live	Get Tasks Live	▼
GET	/tasks/{task_id}	Get Single Task	▼
PATCH	/tasks/{task_id}/state	Manually Update Task State	▼
POST	/tasks/{task_id}/history	Add Task History Update	▼ 
GET	/events	Get Events	▼
GET	/auth/github	Github Login	▼
GET	/auth/github/callback	Github Callback	▼
GET	/connected-users	Get Connected Users	▼
GET	/auth/discord	Discord Login	▼
GET	/auth/discord/callback	Discord Callback	▼
GET	/discord-users	Get Discord Users	▼
GET	/users	Get Users	▼
GET	/discord/activity	Get Discord Activity	▼
GET	/test/standup	Test Standup	▼
GET	/test-daily-standup	Test Daily Standup	▼
Schemas			^
HTTPValidationError > Expand all object			
ValidationError > Expand all object			

5. INDIVIDUAL CONTRIBUTIONS ANALYSIS

5.1 Contributor Distribution

To present an accurate depiction of development weight, initial foundation works have been consolidated into the core engineering profiles. The resulting distribution confirms a clear

division of operational responsibilities between the primary architect and secondary systems engineer.

Table 1: Final Development Contribution Metrics

Contributor	GitHub Identity	Core Focus Areas
Developer 1	Arnav Tripathi	Architecture, Database Migrations, API Routing
Developer 2	Sarvyagya Prakash	Initial Scaffolding, Testing Frameworks, Proxy Integration

5.2 Arnav — Lead Architect

Operational Summary:

Arnav provided strategic engineering leadership, establishing the project's foundational architecture, managing relational database migrations.

- **Relational Database Migration:** Orchestrated the shift to persistent PostgreSQL infrastructure. Refactored domain schemas within `models_sql.py` and modified operational endpoints to eliminate legacy attributes like `updated_at`, aligning data layouts with long-term storage requirements.
- **Task History Subsystem:** Designed and deployed the historical tracking framework. Introduced dedicated updates endpoints (feat: standardise task history structure and add updates endpoint), permitting complete visibility into workflow lifecycle progressions.
- **CORS and Security Integration:** Integrated global Cross-Origin Resource Sharing middleware into `main.py` (fix: add cors middleware), resolving networking hurdles and allowing decoupled frontend client consumers to safely query the API layer.
- **Path Sanitization Enhancements:** Rewrote functional path handling logic within `normalizer.py` to correctly parse and forward modified, added, or removed file arrays down to the visualization graph layer.
- **Runtime Environment Stabilization:** Standardized deployment constraints by forcing the cloud environment lock to Python, correcting dependencies to eliminate runtime platform crashes.

5.3 Sarvyagya Prakash — Systems Engineer

Operational Summary:

Sarvyagya guided foundational environment configuration, proxy routing logic, and system validation. Responsibilities cover early component scaffolding, building the test infrastructure, and linking the service to external graph visualizers.

- **Graph API Proxy Implementation:** Developed the communication interface connecting the backend service to external graph engines. Authored the forwarding mechanism inside the `/graph` proxy endpoint to manage secure data traffic flow down to the Orchestra AI visualization API.
- **Repository and Base Scaffolding:** Spearheaded the foundational setup of the repository environment. Configured structural modules (`database.py`, `early models.py`, and `main.py`

skeletons), creating the underlying pattern that accelerated subsequent feature development.

- **Task Scheduling Pipeline:** Developed scheduling routines inside scheduler.py (6,636 bytes) to control workflow task queuing and background worker execution blocks.
- **Comprehensive Test Suites:** Designed and built the project's quality assurance framework. Authored test_normalizer.py, created webhook integration tests (test_webhook.py), and configured PowerShell automated testing scripts (test_history.ps1) to check state transition stability.

6. PROGRESSION MATRIX

Week 1: Base Architectural Scaffolding

- **Achievements:** Repository initialization and module mapping. Creation of structural data components by Sarvyagya Prakash.
- **Milestones:** Completed initial application scaffolding, established early dependency matrices, and defined local runtime guidelines.

Week 2: Operational Feature Development

- **Achievements:** Schema mapping for relational migrations and drafting workflow validation logic within state_machine.py.
- **Milestones:** Introduced automated testing layers; implemented backend task history tracking structures.

Week 3: System Integration and Production Readiness

- **Achievements:** Execution of PostgreSQL migration scripts, implementation of CORS middleware, and fix deployments for file path normalizers.
- **Milestones:** Resolution of 18 pull requests. Production build finalized and running stably on the Render platform.

7. CONCLUSIONS & SYSTEM ASSESSMENT

- **Clear Technical Distribution:** A balanced division of work where Arnav focuses on system core architecture and persistence, while Sarvyagya owns structural preparation, interface proxies, and code verification.
- **Continuous Integration Discipline:** Constant usage of automated PR builds minimized long-term development drift.
- **Production Readiness:** The repository shows exceptional stability, featuring zero open bugs, passing builds, and active hosting environments.

8. FUTURE IMPLEMENTATIONS AND RECOMMENDATIONS

Core Backend Persistence (Arnav): Eliminate volatile disk I/O and server errors on Render. Inject CORSMiddleware in main.py to fix frontend blocks. Refactor update_task_status() and history endpoints to write directly to PostgreSQL via SQLAlchemy TaskTable. Migrate user identity states from JSON files into persistent ConnectedUserTable and DiscordUserTable models in models_sql.py. Add an optional project_id parameter to GET /tasks for frontend workspace filtering, and delete the redundant, self-referential GET /tasks/live route.

Graph Subsystem & Telemetry Audit Sarvyagya Prakash): Resolve cross-repository data staleness between databases. Implement a thread-safe syncto_graph() worker using asyncio.to_thread() to forward task status updates via authenticated PATCH requests to the Neo4j API layer inside state_machine.py. Conduct an end-to-end multi-database integration

test using commit identifiers (e.g., "fixes T001"). Validate Discord analytics ingestion channels, and execute a data validation audit on GET /events within normalizer.py to ensure strict ISO timestamps, valid platform tracking, and real GitHub handles.

END OF REPORT